

GrafoRotas

Modelagem e análise de rotas rodoviárias entre municípios do estado de São Paulo com Teoria dos Grafos e dados abertos do IBGE e do OpenStreetMap

Nome do Integrante	E-mail (com RA)
Guilherme Haddad Borro	RA 10427699 — [e-mail]
Rafael Lima	RA 10425819 — [e-mail]
Pedro Augusto Yoshikuni	RA 10410287 — [e-mail]
Tiago Silveira Lopes	RA 10417600 — [e-mail]

São Paulo — 2026

Resumo

Este relatório apresenta os resultados parciais do projeto GrafoRotas, cujo objetivo é modelar a malha de deslocamentos rodoviários entre municípios do estado de São Paulo como um grafo e, a partir dessa modelagem, apoiar a análise de rotas e distâncias. Cada município é representado por um vértice e cada ligação entre duas cidades por uma aresta, ponderada pela distância rodoviária real entre elas. Adotou-se um grafo não orientado com peso na aresta (categoria 2). Nesta etapa foi construída a infraestrutura computacional do grafo — representação por lista de adjacência, leitura e gravação em arquivo, inserção e remoção de vértices e arestas, exibição, verificação de conexidade e construção do grafo reduzido — e uma base de dados com [N] municípios e [M] ligações. Os dados são oficiais e reproduzíveis: a lista de municípios provém da Região Geográfica Intermediária de São Paulo (IBGE), as coordenadas provém do OpenStreetMap e o peso de cada aresta é a distância rodoviária real obtida do serviço de roteamento OSRM (OpenStreetMap). Os testes realizados confirmam que a estrutura preserva a consistência do grafo após todas as operações e que a malha modelada é conexa. O problema central do projeto é o do caminho de menor distância entre cidades, a ser resolvido pelo algoritmo de Dijkstra na etapa final.

1. Introdução

1.1 Contextualização

O deslocamento entre cidades é um problema cotidiano e economicamente relevante: transportadoras, serviços de entrega, empresas de turismo e o próprio poder público precisam decidir por quais caminhos ligar um conjunto de localidades gastando o mínimo possível de distância, tempo ou combustível. O estado de São Paulo, o mais populoso e com a malha rodoviária mais densa do país, é um cenário natural para esse tipo de estudo: são centenas de municípios interligados por rodovias, e as decisões de rota têm impacto direto em custo logístico e emissão de poluentes. Problemas desse tipo — encontrar o menor caminho entre dois pontos, conectar todas as cidades com o menor custo total ou planejar um roteiro que passe por várias cidades — são naturalmente descritos e resolvidos pela Teoria dos Grafos.

1.2 Justificativa

A escolha de modelar o problema como um grafo se justifica porque a estrutura de vértices e arestas representa de forma direta e fiel a realidade estudada: as cidades são os vértices e as estradas que as ligam são as arestas, com a distância como peso. Sobre essa representação incide o algoritmo de Dijkstra, que

resolve o problema central do projeto — o cálculo do caminho de menor distância entre duas cidades — de forma eficiente e comprovadamente correta. Além do valor prático, o projeto tem valor acadêmico: exercita a modelagem de um problema real, a implementação da estrutura de dados e a aplicação dos algoritmos vistos em aula. Do ponto de vista social e ambiental, rotas mais curtas tendem a reduzir a distância total percorrida e, com isso, o custo do deslocamento; é importante ressaltar, porém, que a redução de emissões de CO₂ não decorre diretamente da menor distância, pois depende de fatores adicionais como consumo do veículo, condições de tráfego e tipo de combustível, que não são tratados nesta etapa.

1.3 Objetivo

O objetivo geral é desenvolver uma aplicação que modele a malha de municípios do estado de São Paulo como um grafo ponderado e determine o caminho de menor distância entre cidades. O problema central adotado é o do caminho mínimo, a ser resolvido pelo algoritmo de Dijkstra; a construção da árvore geradora mínima (Prim/Kruskal) é tratada como análise complementar, voltada a avaliar o custo de conexão de toda a malha, e não como rota entre dois pontos. Nesta etapa (Parte 2), os objetivos específicos são: (i) coletar os dados das cidades e das distâncias entre elas; (ii) modelar o problema como um grafo não orientado com peso na aresta; (iii) implementar a infraestrutura computacional do grafo, com persistência em arquivo e as operações básicas de manipulação e análise; e (iv) validar a estrutura por meio de testes, deixando a base preparada para a implementação do algoritmo de caminho mínimo na etapa seguinte.

2. Fundamentação Teórica

Um grafo é um par $G = (V, E)$, em que V é um conjunto de vértices e E é um conjunto de arestas que ligam pares de vértices. Quando as arestas não têm direção, o grafo é dito não orientado — a ligação entre A e B é a mesma que entre B e A —, o que é adequado para representar estradas percorráveis nos dois sentidos. Quando cada aresta possui um valor associado, o grafo é ponderado; neste projeto o peso é a distância entre as cidades. Essa combinação corresponde à categoria “grafo não orientado com peso na aresta”.

O grau de um vértice é o número de arestas incidentes a ele — no contexto do projeto, quantas cidades estão diretamente ligadas àquela. Um grafo é conexo quando existe pelo menos um caminho entre qualquer par de vértices; verificar a conexidade é essencial para garantir que se pode ir de qualquer cidade a qualquer outra. Uma componente conexa é um subconjunto maximal de vértices mutuamente alcançáveis, e o grafo reduzido é a contração de cada componente em um único supervértice, útil para diagnosticar se a malha está integralmente conectada.

Para armazenar o grafo em memória, as duas representações usuais são a matriz de adjacência (uma tabela $V \times V$) e a lista de adjacência (para cada vértice, a lista de seus vizinhos). Como a malha rodoviária é esparsa — cada cidade se liga a poucas outras, e não a todas —, a lista de adjacência é mais econômica em memória e mais eficiente para percorrer a vizinhança, motivo pelo qual foi a escolhida. Sobre essa estrutura operam os algoritmos do projeto: a busca em profundidade (DFS), empregada nesta etapa para determinar a conexidade, e, na etapa final, o algoritmo de Dijkstra para o caminho mínimo entre duas cidades — o problema central. A árvore geradora mínima (Prim ou Kruskal) é um recurso complementar: ela minimiza o custo total para conectar todas as cidades da malha, e não a rota entre um par de cidades — são problemas distintos, e o projeto os trata como tais.

3. Trabalhos Relacionados

Aplicações de navegação como o Google Maps e o Waze resolvem, em essência, o mesmo problema tratado aqui: modelam a malha viária como um grafo ponderado e aplicam variações do algoritmo de Dijkstra e do A* para encontrar a melhor rota entre dois pontos, usando distância e tempo estimado como peso. A diferença de escala é grande — trabalham com milhões de segmentos de via e dados de tráfego em tempo real —, mas o núcleo conceitual é o mesmo deste projeto.

O projeto OSRM (Open Source Routing Machine), construído sobre os dados abertos do OpenStreetMap, é um roteirizador de código aberto que também representa a rede de estradas como um grafo e calcula

caminhos mínimos com técnicas de pré-processamento (contraction hierarchies). Ele mostra como a mesma modelagem em grafos escala para redes reais e serve de referência para a estrutura de dados adotada.

No campo da logística, o problema do caixeiro viajante (TSP) e o problema de roteamento de veículos (VRP) são amplamente estudados para planejar entregas que passam por várias cidades minimizando a distância total. Trabalhos como o estudo computacional de Applegate et al. sobre o TSP demonstram tanto a dificuldade quanto as soluções aproximadas do problema — abordagem que o GrafoRotas pretende explorar na etapa final para gerar roteiros entre municípios paulistas.

4. Metodologia

O projeto foi organizado em etapas incrementais. A primeira etapa consistiu na coleta dos dados, feita por um script automatizado e reprodutível (`coletar_dados.py`): a lista oficial de municípios é obtida da API do IBGE, as coordenadas de cada município são obtidas do OpenStreetMap (serviço Nominatim) e a distância rodoviária real de cada ligação é obtida do serviço de roteamento OSRM, também do OpenStreetMap. A segunda etapa foi a modelagem do problema como grafo, definindo o que seriam vértices, arestas e pesos, e a categoria do grafo. A terceira etapa, foco desta entrega, foi a implementação da infraestrutura computacional: a estrutura de dados do grafo e as operações de leitura/gravação em arquivo, manipulação de vértices e arestas, exibição e análise de conexidade. A quarta etapa, planejada para a Parte final, será a implementação do algoritmo de caminho mínimo (Dijkstra) e da árvore geradora mínima como análise complementar. A opção pelo OpenStreetMap dispensa serviços comerciais; a API do Google Maps permanece como alternativa opcional, cujo uso dependeria da verificação prévia de acesso, limites e custos.

A linguagem escolhida foi Python, pela clareza de sintaxe e pela facilidade de integração com APIs externas. O código foi estruturado em módulos independentes — a base de dados das cidades, a classe do grafo, o módulo de integração com o Google Maps e o roteiro de testes — de modo que cada parte possa evoluir de forma isolada. Os testes foram conduzidos por um script que exercita todas as operações e verifica que os totais de vértices e arestas se mantêm consistentes após cada inserção e remoção.

5. Resultados Parciais

5.1 Coleta dos dados

Observação: os valores numéricos assinalados entre colchetes ([N], [M] etc.) são preenchidos automaticamente ao executar o script de coleta (`coletar_dados.py`), pois dependem do resultado da consulta às bases oficiais.

A delimitação do universo de estudo seguiu um critério objetivo e oficial: foram considerados os municípios da Região Geográfica Intermediária de São Paulo, conforme a regionalização do IBGE de 2017. Trata-se de uma unidade territorial oficial, contígua e centrada na Região Metropolitana de São Paulo — o maior polo de deslocamento rodoviário do país —, o que a torna adequada ao problema estudado e evita uma seleção arbitrária de cidades. Esse recorte reúne [N] municípios, quantidade superior ao mínimo de 80 vértices exigido pelo projeto.

A coleta é integralmente automatizada e reprodutível, executada pelo script `coletar_dados.py`, e usa exclusivamente fontes oficiais e abertas. A lista de municípios é obtida diretamente da API de localidades do IBGE. As coordenadas geográficas (latitude e longitude) de cada município são obtidas do OpenStreetMap, por meio do serviço de geocodificação Nominatim. A distância rodoviária real de cada ligação é obtida do OpenStreetMap por meio do serviço de roteamento OSRM, que calcula o trajeto de carro entre os dois municípios. Dessa forma, o peso das arestas passa a ser a distância efetiva pela estrada — e não mais uma aproximação em linha reta.

Foi necessário definir objetivamente o que constitui uma ligação (aresta) entre dois municípios. Adotou-se o seguinte critério: existe uma aresta entre duas cidades quando uma delas está entre as cinco geograficamente mais próximas da outra, sendo o peso dessa aresta a distância rodoviária real fornecida pelo OpenStreetMap. Esse critério de vizinhança reproduz o comportamento da malha rodoviária, em que

municípios próximos tendem a possuir ligação direta, e garante um grafo conexo. O resultado é um grafo com $[N]$ vértices e $[M]$ arestas.

5.2 Modelagem do problema como grafo

A modelagem adotada é a seguinte:

- Vértices: cada vértice representa um município do estado de São Paulo, identificado por um número e rotulado com o nome da cidade.
- Arestas: cada aresta representa a ligação entre duas cidades e é não orientada, pois a estrada é percorrível nos dois sentidos.
- Pesos: cada aresta carrega o peso correspondente à distância rodoviária real entre as duas cidades, em quilômetros, obtida do OpenStreetMap.
- Categoria do grafo: não orientado com peso na aresta (Valor 2).

Uma decisão de modelagem merece justificativa explícita: a opção por um grafo não orientado. Adotá-lo equivale a assumir que a distância entre duas cidades é simétrica, isto é, que ir de A para B custa o mesmo que ir de B para A. Para distâncias rodoviárias entre municípios, essa é uma aproximação razoável, pois eventuais diferenças de sentido tendem a ser pequenas frente à distância total do trajeto intermunicipal. Trata-se, portanto, de uma abstração consciente, escolhida por simplificar a estrutura e por ser plenamente compatível com o algoritmo de Dijkstra. O serviço de roteamento fornece a distância rodoviária do trajeto entre o par de municípios, e esse valor é adotado como peso da aresta não orientada. Vale registrar que a distância rodoviária real pode, em certos trechos, ser assimétrica (mãos únicas, restrições viárias); caso essas assimetrias se mostrem relevantes, a infraestrutura já suporta a migração para um grafo orientado com peso na aresta (Valor 6) sem reescrita — a lista de adjacência armazena o peso de cada sentido de forma independente.

Esboço da modelagem: cidades (vértices) e distancias (arestas ponderadas)

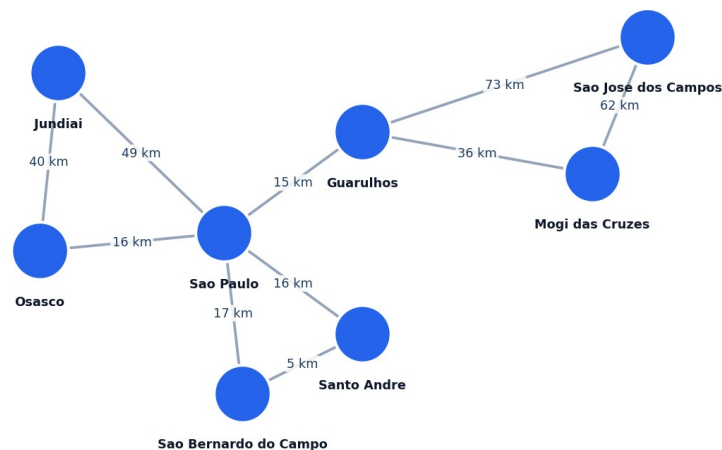


Figura 1 — Esboço da modelagem com poucos vértices: cidades como vértices e distâncias como arestas ponderadas.

A figura a seguir apresenta a malha completa modelada nesta etapa, com os municípios posicionados por suas coordenadas geográficas reais e as ligações entre eles. O tamanho de cada vértice é proporcional ao seu grau (número de cidades diretamente ligadas), o que permite identificar visualmente os municípios mais conectados da região.

[Figura 2 — a malha completa, com os $[N]$ municípios posicionados por suas coordenadas reais e as $[M]$ ligações, é gerada pelo script `gerar_imagens.py` após a coleta dos dados e inserida aqui.]

5.3 Infraestrutura implementada

O grafo é representado por uma lista de adjacência implementada com dicionários aninhados: para cada vértice, um dicionário associa cada vizinho ao peso da respectiva aresta. Essa escolha permite consultar, inserir e remover arestas em tempo médio constante e percorrer a vizinhança de um vértice de forma eficiente. Sobre essa estrutura foram implementadas todas as funcionalidades exigidas nesta etapa:

- Leitura e gravação do arquivo grafo.txt, garantindo a persistência do grafo entre execuções.
- Inserção e remoção de vértices, com remoção automática de todas as arestas incidentes ao vértice retirado.
- Inserção e remoção de arestas, mantendo a simetria própria de um grafo não orientado.
- Exibição do grafo em formato textual (lista de adjacência com nomes das cidades e distâncias).
- Determinação da conexidade por busca em profundidade, com identificação das componentes conexas.
- Construção do grafo reduzido, contraindo cada componente conexa em um supervértice.

O arquivo grafo.txt segue um formato simples e autoexplicativo: a primeira linha traz o número de vértices; as linhas seguintes, o identificador e o nome de cada cidade; em seguida, o número de arestas; e, por fim, cada aresta como um trio identificador de origem, identificador de destino e peso em quilômetros.

5.4 Detalhamento dos resultados obtidos

A execução do roteiro de testes confirmou o funcionamento correto de todas as operações. Os principais resultados são:

- O grafo carregado a partir do arquivo possui [N] vértices e [M] arestas, coincidindo com os dados coletados.
- A verificação de conexidade indicou que o grafo é conexo: existe uma única componente conexa contendo todos os municípios, ou seja, é possível ir de qualquer cidade a qualquer outra pela malha modelada.
- As operações de inserção e remoção preservaram a consistência da estrutura: após inserir e depois remover um vértice de teste, os totais retornaram exatamente aos valores originais; o mesmo ocorreu com a inserção e remoção de arestas.
- A análise da malha reportou o grau mínimo, máximo e médio das cidades e identificou o município mais conectado da região (valores no arquivo saida_execucao.txt do repositório).

Trecho representativo da saída dos testes (preenchido após a execução):

```
Grafo carregado: [N] vertices, [M] arestas.  
O grafo e conexo? SIM  
Numero de componentes conexas: 1  
Grau minimo: [.] | Grau maximo: [.] | Grau medio: [.]
```

5.5 Repositório no GitHub

Todo o conteúdo desenvolvido está organizado em um repositório público no GitHub, contendo este relatório, os arquivos-fonte, o script de coleta de dados (coletar_dados.py) e os arquivos de dados gerados (grafo.txt e cidades.csv). A presença do próprio script de coleta no repositório garante a reprodutibilidade: qualquer pessoa pode executá-lo e obter a mesma base a partir das fontes oficiais. Cada arquivo-fonte traz, em seu cabeçalho, a identificação completa dos integrantes do grupo, uma síntese do conteúdo do arquivo e o histórico de alterações no formato data, autor e breve descrição da atualização, conforme solicitado.

Endereço do repositório: [\[inserir o link do repositório GitHub do grupo\]](#).

5.6 Limitações e ajustes em relação à primeira entrega

A proposta inicial (Parte 1) recebeu avaliação favorável, e os apontamentos então recebidos foram integralmente incorporados nesta etapa. O escopo do objetivo foi delimitado, estabelecendo o caminho mínimo (Dijkstra) como problema central e a árvore geradora mínima como análise complementar, evitando

a diluição em múltiplos algoritmos. A delimitação dos municípios deixou de ser uma seleção ad hoc e passou a seguir um recorte oficial do IBGE (a Região Geográfica Intermediária de São Paulo). A fonte dos dados foi confirmada e passou a ser oficial e reprodutível — IBGE para os municípios e OpenStreetMap para coordenadas e distâncias rodoviárias —, sem depender de serviços comerciais; o Google Maps ficou como alternativa opcional. O critério de existência de uma ligação foi explicitado, e a escolha do grafo não orientado foi justificada, com ressalva sobre a possível assimetria das distâncias reais. Por fim, a justificativa ambiental foi ajustada para não associar diretamente a redução de distância à redução de emissões, que depende de outros fatores.

A principal limitação atual é que o critério de ligação (vizinhança geográfica) é uma aproximação da malha rodoviária real; um refinamento possível é derivar as adjacências diretamente da topologia de estradas do OpenStreetMap. O planejamento da próxima etapa compreende: a implementação do algoritmo de Dijkstra para o caminho mínimo entre cidades — o problema central — e a análise complementar da árvore geradora mínima, ambos operando sobre a base de distâncias rodoviárias reais já coletada nesta etapa.

6. Referências

- APPLEGATE, D. L.; BIXBY, R. E.; CHVÁTAL, V.; COOK, W. J. The Traveling Salesman Problem: A Computational Study. Princeton: Princeton University Press, 2006.
- BONDY, J. A.; MURTY, U. S. R. Graph Theory. New York: Springer, 2008.
- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Algoritmos: Teoria e Prática. 3. ed. Rio de Janeiro: Elsevier, 2012.
- DIJKSTRA, E. W. A note on two problems in connexion with graphs. Numerische Mathematik, v. 1, p. 269-271, 1959.
- GOLDBARG, M. C.; GOLDBARG, E. F. G. Grafos: conceitos, algoritmos e aplicações. Rio de Janeiro: Elsevier, 2012.
- IBGE — Instituto Brasileiro de Geografia e Estatística. Cidades e Estados — São Paulo. Disponível em: <https://www.ibge.gov.br/cidades-e-estados/sp>. Acesso em: set. 2026.
- GOOGLE. Distance Matrix API — Documentation. Disponível em: <https://developers.google.com/maps/documentation/distance-matrix>. Acesso em: set. 2026.
- OPENSTREETMAP FOUNDATION. Open Source Routing Machine (OSRM). Disponível em: <http://project-osrm.org>. Acesso em: set. 2026.